

FEBAUTO Racing

Lab: ROS 2 onboarding in the FEB devkit

Session 1	The devkit, the workspace, packages, nodes, topics, services
Session 2	A publisher/subscriber pair, a turtlesim controller, a patrol service
Source	`docs/lab-ros-onboarding.md`; starter code in `labs/onboarding/`

Formula Electric at Berkeley
September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing
https://febautonomous.github.io/feb-racing/

Contents

1 Setup, once 2

2 What is ROS 2, in one page 2

3 The devkit: our container 3

4 The workspace 3

5 Anatomy of a package 4

6 Creating and building packages 4

7 Nodes, topics and services 5

8 A publisher and a subscriber 5

9 Write your own pair, with a custom message 6

10 A controller for turtlesim (Lab 2, part 1) 7

11 Services: a patrol server (Lab 2, part 2) 8

12 Keep your code in git 9

13 When something is wrong 9

About this lab

One lab, two sessions, adapted from EECS C106A Labs 1 and 2 (Spring 2026) to run in our devkit instead of the course's instructional accounts and distrobox container. Every command below was run as written, in order, on 2026-10-16 in the box. Do them in order. The commands are the same on Mac, Windows and Linux: everything ROS happens inside the container, and the launcher fills in the ports and paths for your machine.

By the end you can:

- start the devkit, open a shell in it and see its desktop in your browser;
- create a workspace package with its dependencies, build it with `colcon`, run nodes with `ros2 run`;
- inspect nodes, topics and services with the `ros2` tools and `rqt_graph`;
- write a publisher/subscriber pair with a custom message type;
- write a service server and client, and a node that drives an existing one (turtlesim).

Starter code is in `labs/onboarding/` of the repo: `chatter` (a talker and a listener), `turtle_patrol_interface` (the `Patrol` service definition) and `turtle_patrol` (its server and client). Nothing else is needed. You type the rest.

1 Setup, once

You need Docker Desktop (Mac, Windows) or Docker Engine (Linux), git, and about 8 GB of disk.

```
mkdir ~/FEB && cd ~/FEB
git clone https://github.com/FEBAutonomous/feb-racing.git && cd feb-racing
./feb setup --name "Your Name"
```

Already have the clone? `./feb update` instead: it pulls the repo, the newer image and the newer simulator app in one go.

`setup` checks Docker, pulls the image, downloads the simulator app (not used in this lab) and creates your box. `docs/devkit.md` is the one-page reference for it. On a Mac, keep `~/FEB` under `/Users`: Docker can only mount folders from there.

2 What is ROS 2, in one page

ROS 2 is a set of libraries and tools for robot software. A robot's software is split into small programs called **nodes** (a lidar driver, a planner, a motor controller), each doing one job. Nodes find each other over the network with no central server and talk in three ways:

- **Topics**: a node *publishes* messages on a named topic, any number of nodes *subscribe*. A stream, asynchronous, like a broadcast. The lidar publishes `/scan`; the planner subscribes.
- **Services**: one node *calls* another and waits for the answer. One request, one response, like ordering at a counter. "Spawn a turtle at (2, 2)."
- **Parameters**: a node's settings, readable and writable at run time.

A **message** is a typed data structure (a `String`, a `Twist` with linear and angular velocity). Nodes can be written in Python (`rclpy`) or C++ (`rclcpp`) and mix freely. Our racing stack is exactly this: a bridge node publishes the simulator's lidar and encoders as topics, the racer node subscribes, and it publishes steering and throttle topics back.

3 The devkit: our container

The course runs ROS 2 in a distrobox container on the lab machines. We run it in a Docker container on your laptop. Same idea, the "sealed lunchbox": Ubuntu, ROS 2 Humble, all our dependencies and tools are inside; your code stays on your laptop and is mounted in. The box keeps what you install in it and is there every time you come back.

Open a terminal in it, with the turtle simulator and its desktop in your browser:

```
./feb shell --gui --apps turtlesim
```

What happens: your browser opens `http://localhost:6080/vnc.html?autoconnect=1&resize=scale`, the box's screen, with a turtle in a blue window, and your terminal is now **inside the box**. If the tab did not open, paste that URL.

Your prompt now reads `box:~/FEB/feb-racing$`. The commands you will use from your laptop's own terminal, from the repo folder:

Command	What it does
`./feb shell`	a terminal inside the box; open as many as you like
`./feb shell --gui`	the same, and the box's desktop in a browser tab
`./feb shell --gui --apps "turtlesim rqt"`	...with those windows opened for you
`./feb code`	VS Code attached to the box, its terminals inside
`./feb reset`	wipe the box and start fresh; your files stay
`./feb up --track loop`	later: the simulator, for the racing labs

You never type a `docker` command. `docs/devkit.md` shows what each of those does underneath. Two habits:

- A terminal inside the box is one you opened with `./feb shell` or in VS Code after `./feb code`. `ros2` and `colcon` only exist there. If a command says `ros2: command not found`, you are in a laptop terminal.
- Anything with a window (`rqt_graph &`, `rviz2 &`) opens on the desktop in the browser tab.

4 The workspace

Inside the box, run `pwd` and `ls`:

```
/home/feb/FEB/feb-racing
README.md  docs  feb  feb-sim  labs  stack  starter_kit  tracks  ...
```

That is your clone, the same folder as on your laptop: `/home/feb/FEB` in the box is `~/FEB` on the laptop, mounted, not copied. A file you save on the laptop is there at once, and a package you create inside the box appears on the laptop at once, owned by you.

`stack/` is your ROS workspace: the place `colcon` builds from. It is the same layout the course lab describes, with the packages directly under the workspace instead of under `src/`:

- `stack/<package>/` is the source of each package you make.
- `stack/build/` is where packages are compiled: the prep station.
- `stack/install/` is the result: executables, message code, and the `setup.bash` that tells ROS where it all is. Sourcing it is how ROS finds *your* packages.
- `stack/log/` is what every build printed, in case something fails.

All of it is on your laptop and survives anything you do to the box. The simulator bridge, the car's drivers and our house nodes are a second workspace inside the image, `/home/autodrive_devkit`, already built and already sourced in every box terminal.

5 Anatomy of a package

A **package** is the unit of ROS code: a folder with a `package.xml` describing it and its dependencies, plus source, launch files, message definitions. Look at the one we will play with:

```
cd /opt/ros/humble/share/turtlesim
ls
cat package.xml
```

`package.xml` names the package, its maintainers and licence, and what it depends on: `<build_depend>` at build time, `<exec_depend>` at run time, `<depend>` both. `rclcpp` there is the C++ client library; Python packages depend on `rclpy` instead, and that is what we use.

Tools that answer questions about packages (`--help` on any of them lists the options):

```
ros2 pkg prefix turtlesim                          # where a package is installed
ros2 pkg executables turtlesim                     # the programs it provides
ros2 interface show example_interfaces/msg/String  # the fields of a message type
ros2 pkg prefix autodrive_roboracer                # our simulator bridge, built into
    the image
```

6 Creating and building packages

Every package you make goes in `stack/` (anywhere outside `~/FEB` is the box's own disk and goes with a reset). Create two: a bare one and one with dependencies declared.

```
cd ~/FEB/feb-racing/stack
ros2 pkg create foo --build-type ament_python
ros2 pkg create bar --build-type ament_python --dependencies rclpy std_msgs
    geometry_msgs turtlesim
ls foo
grep depend bar/package.xml
```

`ament` is ROS 2's build system; `ament_python` is a pure Python package, `ament_cmake` a C++ one (or one that only holds message definitions, see section 9). In `foo` you find `package.xml` (metadata, dependencies), `setup.py` (how to install the Python code, and where executables are registered), `setup.cfg`, `resource/foo` (an empty marker file), `foo/` (the Python module your nodes go in, with an empty `__init__.py`) and `test/` (style checks, ignore them). Open them on your laptop: they are at `feb-racing/stack/foo`.

Build from the workspace root, not from inside the package:

```
cd ~/FEB/feb-racing/stack
echo $AMENT_PREFIX_PATH
colcon build --symlink-install --packages-select foo bar
source install/setup.bash
echo $AMENT_PREFIX_PATH
ros2 pkg prefix bar
```

`--packages-select` builds just the ones you name (a plain `colcon build` builds everything in `stack/`, which is fine too). `--symlink-install` links the installed Python files to your source, so an edit takes effect on the next `ros2 run` without rebuilding; you rebuild only when you add a package, an executable or a message. `source install/setup.bash` adds your packages to the paths ROS searches, which is the difference between the two `echo` lines. Every new shell needs it (a new `./feb shell` does it for you).

Checkpoint 1. Be able to explain the workspace layout and what `build`, `install`, `log` and `stack/` are; show the build; explain `package.xml`; find a package's path.

7 Nodes, topics and services

turtlesim is already running on the desktop (section 3). Ask ROS what is there:

```
ros2 node list
ros2 node info /turtlesim
```

`/turtlesim` is the node. It subscribes to `/turtle1/cmd_vel` (velocity commands), publishes `/turtle1/pose`, and serves `/spawn`, `/clear`, `/kill` and more. (`/foxglove_bridge` is the devkit's own node that lets the Foxglove app on your laptop see topics; ignore it.)

Drive the turtle. In a **second** terminal on your laptop:

```
./feb shell
ros2 run turtlesim turtle_teleop_key
```

Click into that terminal and use the arrow keys: the turtle in the browser moves. This is `ros2 run <package> <executable>`: the teleop node reads your keys and publishes `Twist` messages on `/turtle1/cmd_vel`; the turtlesim node subscribes and moves the turtle.

See the graph. Leave the teleop running, and in a third terminal (`./feb shell`):

```
rqt_graph &
```

It opens on the browser desktop next to the turtle (drag windows by their title bar; the desktop is 1600 by 900). Press the refresh button (the circular arrow, top left): `rqt_graph` does not update on its own, and its first picture is usually incomplete. You then see two ovals, one arrow, one topic. Under *Hide*, untick *Debug* and refresh again: the terminals listening in appear too.

Topics by hand:

```
ros2 topic list
ros2 topic list -v                      # with message types and counts
ros2 topic type /turtle1/cmd_vel
ros2 topic echo /turtle1/pose           # Ctrl+C to stop; drive the turtle and watch
ros2 topic echo /turtle1/cmd_vel
```

Services are the other pattern: one request, one response.

```
ros2 service list
ros2 service type /clear
ros2 service call /clear std_srvs/srv/Empty "{}"
```

The turtle's track is wiped. `/spawn` takes arguments; find out which:

```
ros2 service type /spawn
cat $(ros2 pkg prefix turtlesim)/share/turtlesim/srv/Spawn.srv
```

Above the `--` is the request (x, y, theta, name), below it the response (the name given).

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 1.2, name: '
    new_turtle'}"
```

A second turtle appears, and `ros2 topic list` now has `/new_turtle/...` topics.

Checkpoint 2. Explain node, topic, message, service; drive the turtle; show a topic's type and echo it; spawn a turtle.

8 A publisher and a subscriber

The starter package `chatter` is in the repo under `labs/`, which is not mounted into the container, so copy it on your **laptop**, in the repo folder:

```
cp -r labs/onboarding/chatter stack/
```

Then inside the container:

```
cd ~/FEB/feb-racing/stack
colcon build --symlink-install --packages-select chatter
source install/setup.bash
ros2 pkg executables chatter
```

Two executables, `talker` and `listener`. Run the first in this terminal and the second in another (`./feb shell`):

```
ros2 run chatter talker
ros2 run chatter listener
```

The talker prints `Publishing: "Hello World: n"` twice a second; the listener prints `I heard:` with the same text. Start a second listener: both hear every message. Start a second talker: the listener hears both, interleaved. Ctrl+C stops each.

Read `stack/chatter/chatter/publisher_member_function.py` and `subscriber_member_function.py` on your laptop. A node is a class that inherits `rclpy.node.Node`; `create_publisher(type,topic,depth)` and `create_subscription(type,topic,callback,depth)` make the ends; `create_timer(seconds, callback)` runs something regularly; `rclpy.spin(node)` keeps the node alive, running its timers and callbacks until Ctrl+C.

Now `stack/chatter/setup.py`, at the bottom:

```
entry_points={
    'console_scripts': [
        'talker = chatter.publisher_member_function:main',
        'listener = chatter.subscriber_member_function:main',
    ],
},
```

This is what makes `ros2 run chatter talker` exist: executable name, then `module:function`. The file can be called anything; the executable name is what you declare here. Without an entry point there is nothing for `ros2 run` to run.

9 Write your own pair, with a custom message

Build a talker that publishes what you type, with a timestamp, and a listener that prints it with the time it arrived. Read this whole section first.

The message type. `std_msgs/String` has one field; you need two (text and a timestamp). Custom messages are inconvenient in an `ament_python` package, so the convention is a separate `ament_cmake` *interface package* that holds only message and service definitions:

```
cd ~/FEB/feb-racing/stack
ros2 pkg create my_chatter_msgs --build-type ament_cmake
mkdir my_chatter_msgs/msg
```

Create `my_chatter_msgs/msg/TimestampString.msg` with two lines, one field each: a `string` and an `int64`. Then tell the build about it. In `my_chatter_msgs/package.xml`, after the existing `<buildtool_depend>` line, add:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

In `my_chatter_msgs/CMakeLists.txt`, before `ament_package()`, add:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/TimestampString.msg"
```

```
)
find_package(rosidl_default_runtime REQUIRED)
ament_export_dependencies(rosidl_default_runtime)
```

Build and check ROS can see it:

```
cd ~/FEB/feb-racing/stack
colcon build --symlink-install --packages-select my_chatter_msgs
source install/setup.bash
ros2 interface show my_chatter_msgs/msg/TimestampString
```

The nodes. Create `my_chatter` (`ament_python`, dependencies `rclpy my_chatter_msgs`) in `stack/`, and write two files in its module folder, using `chatter` as your model:

- `my_talker.py`: in a loop, `input()` a line from the terminal, make a `TimestampString` with the text and `self.get_clock().now().nanoseconds`, publish it on `/user_messages`. Import the type with `frommy_chatter_msgs.msgimportTimestampString`.
- `my_listener.py`: subscribe to `/user_messages` and print `Message:<text>,Sentat:<stamp>,Receivedat:<stamp>`, where the second stamp is the listener's own clock when the callback ran. It is not in the message; think about why it can differ from the first.

Add entry points `my_talker` and `my_listener` to `setup.py`, build with `--packages-select my_chatter`, source, and run each in its own `./feb shell`:

```
ros2 run my_chatter my_talker
ros2 run my_chatter my_listener
```

Type lines into the talker; they appear in the listener with both stamps.

Checkpoint 3. Explain everything in `stack/`; show the message definition and that both packages build; demonstrate the pair.

10 A controller for turtlesim (Lab 2, part 1)

Replace `turtle_teleop_key` with your own node. Create `lab2_turtlesim` (`ament_python`, dependencies `rclpy std_msgs geometry_msgs turtlesim`). Its node `turtle_controller`:

1. Takes the turtle's name on the command line:

   ```
   ros2 run lab2_turtlesim turtle_controller <name_of_turtle>
   ```

   Use `sys.argv`, after `rclpy.utilities.remove_ros_args(sys.argv)` strips ROS's own flags.

2. Publishes `geometry_msgs/msg/Twist` on that turtle's `cmd_vel` topic when the user presses keys. Reading single keystrokes from a terminal is fiddly; `input()` plus Enter is fine (`w/a/s/d`, say). Bonus if you capture keys directly.

3. Works for several turtles at once: spawn more with `/spawn` (section 7) and run one controller per turtle in its own `./feb shell`, each with a different name.

One thing you will meet: a node that publishes within a few hundred milliseconds of starting can lose those first messages, because turtlesim has not yet discovered the new publisher. Typing is slower than that, so it does not matter here, but remember it when a node's first message "disappears".

Checkpoint 4. Explain the package; show it builds; drive two turtles from two controllers.

11 Services: a patrol server (Lab 2, part 2)

A **service** is a request and a response between two nodes: a client sends a request, the server's callback runs and fills in a response, the client gets it. For something you ask for once, on demand; topics are for streams.

Three parts define one: a service type (`.srv` file), a server node, a client node. The starter has all three. On the laptop:

```
cp -r labs/onboarding/turtle_patrol_interface labs/onboarding/turtle_patrol stack/
```

Read `stack/turtle_patrol_interface/srv/Patrol.srv`:

```
float32 vel
float32 omega
---
geometry_msgs/Twist cmd
```

Request above the line, response below. Note in its `CMakeLists.txt` the `DEPENDENCIES geometry_msgs` line: a service that uses another package's message must say so, and `package.xml` must depend on that package.

`stack/turtle_patrol/turtle_patrol/patrol_server.py` offers `/turtle1/patrol`: a request stores a `Twist` with the requested speed and turn rate, and a timer publishes it on `/turtle1/cmd_vel` ten times a second, so the turtle circles. `patrol_client.py` calls it once: `create_client`, `wait_for_service`, `call_async`, then `spin_until_future_complete` to wait for the answer. Build and run (server and client in separate shells):

```
cd ~/FEB/feb-racing/stack
colcon build --symlink-install --packages-select turtle_patrol_interface turtle_patrol
source install/setup.bash
ros2 run turtle_patrol patrol_server
ros2 run turtle_patrol patrol_client
```

turtle1 circles. The service can also be called from the command line, with other values:

```
ros2 service call /turtle1/patrol turtle_patrol_interface/srv/Patrol "{vel: 2.0, omega:
    1.0}"
```

Your extension: one server, many turtles, with teleport. Change the system so that a single server controls any turtle by name, teleports it to a requested pose first, then patrols it:

```
ros2 service call /turtle_patrol turtle_patrol_interface/srv/Patrol \
  "{turtle_name: 'new_turtle', vel: 2.0, omega: 1.0, x: 1.0, y: 3.0, theta: 1.0}"
```

Requirements:

1. One server process, `multi_patrol_server`, serving `/turtle_patrol`. Not one server per turtle. The request's `turtle_name` says which turtle.
2. The request also carries `x`, `y`, `theta`. The server teleports that turtle there with turtlesim's own teleport service, then starts its patrol with `vel` and `omega`.
3. A client, `patrol_client`, called as `ros2runturtle_patrolpatrol_client<name><x><y> <theta><vel><omega>`, that fills the request from those arguments, calls the service and prints the response.

What to change: the `.srv` (add `string turtle_name`, `float32 x`, `float32 y`, `float32 theta` to the request; add to the response, for instance `bool success` and `string message`, but keep `geometry_msgs/Twist cmd` or the starter server and client stop working), then rebuild the interface package before the Python one.

Hints:

- Find the teleport service with `ros2servicelist`, then its type with `ros2servicetype<name>` and its fields with `ros2interfaceshow<type>`. Your server is a *client* of it; model that part on `patrol_client.py`.
- Keep per-turtle state in a dictionary keyed by name: the publisher for `/<name>/cmd_vel` and the current `Twist`. The timer loops over the dictionary.
- Do not wait for the teleport inside your service callback with `spin_until_future_complete`: the node is already spinning to run your callback, and waiting there deadlocks. `call_async` and carry on; the teleport completes a moment later.

Checkpoint 5. Explain how a service call differs from a subscriber callback; show two turtles patrolling from different starting poses; explain why one server is enough.

12 Keep your code in git

`stack/` is ignored by the `feb-racing` repo on purpose: it is yours. Make it a repo of its own and push it to a **private** GitHub repository.

Once, set up a key (skip if GitHub already knows this laptop):

```
git config --global user.name "Your Name"
git config --global user.email you@berkeley.edu
ssh-keygen -t ed25519 -C "you@berkeley.edu"       # Enter for the default file; a
    passphrase is wise
cat ~/.ssh/id_ed25519.pub                           # paste into GitHub -> Settings ->
    SSH and GPG keys
ssh -T git@github.com                               # "Hi <you>! You've successfully
    authenticated"
```

Create an empty private repository on github.com, then:

```
cd stack
git init
git remote add origin git@github.com:<you>/<repo>.git
git add .
git commit -m "ROS onboarding lab"
git push -u origin main
```

From then on: `git add`, `git commit -m "..."`, `git push`. Partners may share the repository; it stays private.

13 When something is wrong

Symptom	Cause, fix
`ros2: command not found`	you are in a laptop terminal; `./feb shell`
`Package 'x' not found` after building	this shell has not sourced the new build: `source install/setup.bash`
the browser desktop says not connected	reload the tab; still nothing, `./feb reset` and `./feb shell --gui` again
`mounts denied` (Mac)	the clone is outside `/Users`: move it
a package I created is gone after `./feb reset`	it was created outside `~/FEB`; recreate it in `stack/`
my node's first message never arrives	discovery: see section 10
the build fails	run `colcon build` again in `stack/` and read the error

Checkoff summary

1. Workspace, `colcon build`, `package.xml`, `ros2 pkg prefix`.
2. Node, topic, message, service; teleop, echo, spawn.
3. `my_chatter_msgs` and `my_chatter` built and demonstrated.
4. `lab2_turtlesim` driving two turtles.
5. `multi_patrol_server` patrolling two turtles from requested poses; code pushed.